IDS Traffic Monitoring Dashboard - User Guide

A modern, web-based dashboard for monitoring your IDS pipeline in real-time with interactive visualizations.

🎯 Overview

The Streamlit dashboard provides a comprehensive, user-friendly interface to monitor all aspects of your IDS pipeline without relying on terminal output. It displays:

- ⚡ **Latency Metrics**: Real-time latency tracking for each pipeline component
- 🚀 **Throughput Metrics**: Events processed per second by component
- 🤖 **ML Predictions**: Distribution of attack types and benign traffic
- ⚠️ **Errors & Warnings**: System errors categorized by severity
- 💻 **System Resources**: CPU, memory, disk, and network usage

🚀 Quick Start

1. Install Dependencies

```
# Install required packages
pip3 install -r requirements.txt

# Or install manually
pip3 install streamlit plotly pandas
```

2. Start Your IDS Pipeline

Terminal 1 - Start the IDS pipeline:

PROF

```
# AF_PACKET mode (recommended for testing)
sudo ./run_afpacket_mode.sh

# OR DPDK mode (for production)
sudo ./run_dpdk_mode.sh
```

3. Launch the Dashboard

Terminal 2 - Launch the dashboard:

```
./run_dashboard.sh
```

The dashboard will automatically open in your default web browser at `http://localhost:8501`

📊 Dashboard Sections

Overview Metrics

At the top of the dashboard, you'll see four key metrics:

- **Total Events Processed**: Count of all events handled by the pipeline
- **ML Predictions**: Number of predictions made by ML models
- **Average Latency**: Mean latency across all components
- **Total Errors**: Count of errors encountered

Latency Metrics Section

- **Time Series Chart**: Real-time latency trends for each component
- **Distribution Chart**: P50, P95, P99 percentiles comparison
- **Detailed Table**: Expandable statistics table with min/max/mean values

Throughput Metrics Section

- **Time Series Chart**: Events per second over time by component
- **Summary Table**: Total events and average rate per component

ML Predictions Section

- **Pie Chart**: Visual distribution of prediction classes
- **Performance Metrics**: Inference time and confidence scores
- **Breakdown**: Detailed count and percentage for each prediction class

System Resources Section

- **CPU Gauge**: Real-time CPU usage percentage
- **Memory Gauge**: Real-time memory usage percentage
- **I/O Statistics**: Disk and network read/write metrics

PROF

Errors & Warnings Section

- **By Component**: Which components are generating errors
- **By Severity**: Critical, error, or warning classifications

⚙ Dashboard Controls

Sidebar Settings

Auto-refresh Interval

- Range: 1-60 seconds
- Default: 5 seconds
- Controls how often the dashboard updates

Data Lookback Window

- Range: 1-60 minutes
- Default: 10 minutes
- Controls how much historical data to display

Pipeline Status

The sidebar shows:

- ✅ Metrics file status (active/inactive)
- 📊 File size of current metrics file
- 🕐 Last modification time

🎨 Features

Auto-Refresh

The dashboard automatically refreshes at the configured interval, pulling the latest metrics without manual intervention.

Interactive Visualizations

- **Zoom**: Click and drag to zoom into specific time periods
- **Pan**: Drag to move through the timeline
- **Hover**: Hover over data points for detailed information
- **Legend**: Click legend items to show/hide specific components

Responsive Design

- Wide layout optimized for desktop viewing
- Adjusts to different screen sizes
- Print-friendly views available

PROF

📁 File Locations

```
IDS/
├── dashboard.py              # Main dashboard application
├── run_dashboard.sh          # Launcher script
├── requirements.txt          # Python dependencies (updated)
└── logs/
    └── metrics/
        └── metrics_YYYYMMDD.jsonl  # Metrics data source
```

🔧 Advanced Usage

Custom Metrics Directory

If your metrics are stored in a different location:

```
# Via launcher script
./run_dashboard.sh /path/to/custom/metrics

# Or directly with streamlit
streamlit run dashboard.py -- --metrics-dir /path/to/custom/metrics
```

Custom Port

To run on a different port:

```
streamlit run dashboard.py --server.port 8502
```

Remote Access

To access the dashboard from other machines:

```
streamlit run dashboard.py --server.address 0.0.0.0
```

Note: Only do this on trusted networks!

🎯 Usage Scenarios

Scenario 1: Real-Time Monitoring

Use Case: Monitor live traffic during an active incident

PROF

1. Start IDS pipeline
2. Launch dashboard
3. Set auto-refresh to 1-2 seconds
4. Watch metrics update in real-time

Scenario 2: Performance Analysis

Use Case: Analyze pipeline performance over time

1. Set lookback window to 30-60 minutes
2. Generate test traffic
3. Review latency percentiles
4. Check throughput rates
5. Identify bottlenecks

Scenario 3: ML Model Evaluation

Use Case: Evaluate ML model predictions

1. Run diverse traffic patterns
2. Monitor ML Predictions section
3. Check prediction distribution
4. Review confidence scores
5. Analyze inference times

Scenario 4: System Health Check

Use Case: Verify system resources are adequate

1. Check System Resources section
2. Monitor CPU and memory usage
3. Review disk I/O patterns
4. Ensure no resource bottlenecks

🐛 Troubleshooting

Dashboard Shows "Waiting for Metrics"

Problem: No data appears in dashboard

Solutions:

1. Verify IDS pipeline is running: `ps aux | grep suricata`
2. Check metrics file exists: `ls -lh logs/metrics/metrics_$(date +%Y%m%d).jsonl`
3. Verify metrics are being written: `tail -f logs/metrics/metrics_$(date +%Y%m%d).jsonl`
4. Wait 30-60 seconds for first metrics to appear

Dashboard Won't Start

Problem: `streamlit: command not found`

PROF

Solutions:

```
# Verify Python installation
python3 --version

# Install streamlit
pip3 install streamlit plotly pandas

# Or use the launcher which auto-installs
./run_dashboard.sh
```

High Memory Usage

Problem: Dashboard consuming too much memory

Solutions:

1. Reduce lookback window (try 5 minutes instead of 10)
2. Close other browser tabs
3. Restart dashboard periodically
4. Clear browser cache

Metrics Not Updating

Problem: Dashboard shows old data

Solutions:

1. Check auto-refresh is enabled (look at sidebar)
2. Click "🔄 Refresh Now" button
3. Verify IDS pipeline is still running
4. Check metrics file modification time in sidebar

Charts Not Displaying

Problem: Visualizations show errors or blank spaces

Solutions:

1. Check browser console for JavaScript errors (F12)
2. Clear browser cache and reload
3. Try a different browser (Chrome/Firefox recommended)
4. Update plotly: `pip3 install --upgrade plotly`

📈 Best Practices

Performance Optimization

1. **Adjust Refresh Interval**
   - Use 5s for normal monitoring
   - Use 1-2s only during active investigations
   - Use 10-30s for long-term monitoring
2. **Manage Lookback Window**
   - Smaller windows = faster loading
   - Larger windows = more context
   - Balance based on your needs
3. **Browser Tab Management**
   - Keep dashboard in dedicated tab
   - Close unused tabs to save memory
   - Use browser bookmarks for quick access

PROF

Workflow Tips

1. **Multi-Monitor Setup**
    - Terminal on one screen (IDS pipeline)
    - Dashboard on second screen (monitoring)
    - Log files on third screen (troubleshooting)
2. **Keyboard Shortcuts**
    - `R`: Reload dashboard (when in browser)
    - `Ctrl+C`: Stop dashboard (in terminal)
    - `F11`: Full screen mode (in browser)
3. **Screenshot Documentation**
    - Use browser screenshot tools
    - Capture interesting patterns
    - Document incidents with visuals

🔗 Integration with Existing Tools

With Terminal Dashboard

You can run both the old terminal dashboard and new web dashboard simultaneously:

Terminal 1: IDS Pipeline

```
sudo ./run_afpacket_mode.sh
```

Terminal 2: Terminal Dashboard

PROF

```
./monitor_metrics.sh
```

Terminal 3: Web Dashboard

```
./run_dashboard.sh
```

With Log Files

The dashboard reads from the same metrics files as the terminal dashboard:

- `logs/metrics/metrics_YYYYMMDD.jsonl` - JSON Lines format
- Both tools can read concurrently without conflicts

📚 Additional Resources

Related Documentation

- `METRICS_GUIDE.md` - Understanding metrics collection
- `MONITORING_SETUP.md` - Setting up monitoring
- `METRICS_README.md` - API reference for metrics

Example Commands

Check dashboard is running:

```
ps aux | grep streamlit
```

View dashboard logs:

```
# Streamlit logs to terminal where it was launched
```

Stop dashboard:

```
# Press Ctrl+C in the terminal running the dashboard
# Or kill the process
pkill -f "streamlit run dashboard.py"
```

🎓 Understanding the Metrics

Latency Metrics

PROF

- **P50 (Median)**: 50% of operations complete faster than this
- **P95**: 95% of operations complete faster than this
- **P99**: 99% of operations complete faster than this
- Lower values = better performance

Throughput Metrics

- **Events/Second**: Number of events processed per second
- **Total Events**: Cumulative count of processed events
- Higher values = better throughput

ML Metrics

- **Inference Time**: Time taken to make a prediction
- **Confidence**: Model's certainty in its prediction (0-1)
- **Prediction**: Classification result (BENIGN, DDoS, etc.)

System Metrics

- **CPU**: Processor utilization (keep < 80%)
- **Memory**: RAM usage (keep < 90%)
- **Disk I/O**: Read/write operations
- **Network**: Data transmitted/received

🌟 Tips for Effective Monitoring

1. **Establish Baselines**
    - Run dashboard with normal traffic
    - Note typical latency and throughput values
    - Compare against these during incidents
2. **Set Mental Thresholds**
    - P99 latency > 100ms → investigate
    - CPU > 80% → consider scaling
    - Memory > 90% → memory leak?
    - Errors > 10/min → check logs
3. **Watch for Patterns**
    - Sudden latency spikes → bottleneck
    - Throughput drops → upstream issue
    - Error clusters → systematic problem
    - Resource trends → capacity planning
4. **Use Alongside Logs**
    - Dashboard shows "what"
    - Logs show "why"
    - Use both for full picture

PROF

🎉 Success Indicators

Your dashboard is working correctly if you see:

- ✅ Metrics updating every few seconds
- ✅ Latency values in milliseconds (not zero)
- ✅ Throughput showing events processed
- ✅ ML predictions accumulating
- ✅ System resources being tracked
- ✅ Minimal errors (ideally zero)

📞 Getting Help

If you encounter issues:

1. Check this guide's troubleshooting section
2. Review `MONITORING_SETUP.md` for pipeline issues
3. Verify metrics are being generated: `./monitor_metrics.sh --status`
4. Check Streamlit logs in the terminal
5. Ensure all dependencies are installed

Happy Monitoring! 🛡️📊

For questions or issues, refer to the main README.md or other documentation files.